Programming
Human-Centered Artificial Intelligence
Dr. Ramon Winterhalder
A.Y. 2026/27

Exercise sheet 5

Strings – text as a sequence

Not graded, not handed in. Do them anyway. This is where the learning happens! Solutions are published after we discuss them in class.

Write the index numbers out. Before you answer anything in Part A, write the word on paper with 0, 1, 2, … under it, and the negative numbers under that. Every slicing mistake is an off-by-one, and you cannot see an off-by-one without the numbers in front of you.

Part A — Strings as sequences

A1 *Twelve slices*

Throughout this question, `w = "Programming"`.

Give the value of each. Two of them are worth checking twice.

```
1.  w[0]        5.  w[0:3]        9.  w[-3:]
2.  w[4]        6.  w[3:7]       10.  w[::3]
3.  w[-1]       7.  w[:5]        11.  w[::-1]
4.  w[-4]       8.  w[7:]        12.  w[4:4]
```

Number 12 is not an error. Say what it gives and why.

A2 *True or False*

Still with `w = "Programming"`.

```
1.  "gram" in w
2.  w[0] == "p"
3.  len(w) == 11
4.  "Zoo" < "apple"
5.  "abc" < "abd"
6.  w[-1] == w[10]
```

Number 4 surprises most people. Explain it in one sentence.

A3 *Write the slice*

For each description, write the slice that produces it. Use `w` where you can, and `len(w)` nowhere — there is a shorter way.

a) the first three characters;

b) the last three characters;

c) everything except the first character;

d) everything except the last two characters;

e) the middle character (there are 11, so there is exactly one);

f) the whole word, backwards.

A4 *Why does this fail?*

```
word = "cat"
word[0] = "b"
print(word)
```

a) Which error does this raise?

b) Say in one sentence what the error message is telling you.

c) Rewrite it so that `word` ends up as `"bat"`, without assigning to an index.

d) Did the original string `"cat"` change?

A5 *Count something*

Write a program that asks for a line of text and prints how many **digits** it contains.

a) Write the comment plan first.

b) Use a loop over the characters, and `in` to test each one.

c) Check it on `a1b2c3` (answer: 3) and on a line with no digits at all.

Then, in one sentence: which pattern from lecture 4 is this?

Part B — String methods

B1 *What comes back?*

Throughout: `t = "  Ada  Lovelace  "` — two spaces at each end, and two in the middle.

Give the exact result, including any spaces. Write `"like this"` so the spaces are visible.

```
1.  t.strip()                      5.  t.count("a")
2.  t.lower()                      6.  t.find("Lov")
3.  t.upper()                      7.  t.strip().startswith("Ada")
4.  t.replace(" ", "")             8.  t.split()
```

Number 5 has a trap in it. Number 8 gives back something that is not a string — what?

B2 *These do nothing*

Both programs run, raise nothing, and print the wrong thing.

```
# 1                                # 2
name = "ada"                       message = "  hi  "
name.upper()                       message.strip()
print(name)                        print(f"[{message}]")
```

a) What does each one print?

b) What was each one meant to print?

c) Fix both. The fix is the same in each case — say what it is in one sentence.

B3 *It runs, and the count is wrong*

This is supposed to count the vowels in a name.

```
name = input("Name: ")

vowels = 0
for ch in name:
    if ch in "aeiou":
        vowels = vowels + 1

print(vowels)
```

a) What does it give for `Ada Lovelace`?

b) What should it give?

c) Which character is it missing, and why?

d) Fix it by changing one line.

B4 *Clean up a name*

Someone has typed `"  aDA loveLACE  "` into a form.

Write a program that prints it as `Ada Lovelace` — correct capitals, single spaces, no spaces at the ends.

Hint: `.split()` throws away extra spaces, and there is a method that joins things back together with one space between them.

B5 *Initials*

Ask for a full name and print the initials, like this:

```
Full name: ada byron lovelace
A.B.L.
```

It should work for two names or four, not just three.

B6 *Is it a CSV file?*

Write a program that asks for a file name and prints `yes` if it ends in `.csv` and `no` otherwise.

Then check it on all four of these:

```
data.csv        data.CSV        data.csv.txt        csv
```

One of them will probably break your first attempt. Fix it, and say in one sentence what the fix was.

If you finish early

Write a program that counts how many times each letter appears in a piece of text — printing only the letters that actually occur.

You have everything you need: a loop over `"abcdefghijklmnopqrstuvwxyz"`, and `.count()`.

Then think about what is wasteful in that approach: it reads the whole text twenty-six times. In lecture 7 we meet the structure that does it in one pass.